

JobConnect — Complete Project Guide

Online Job Recruitment System · Beginner-friendly review, features checklist, install & run guide
For students / new developers who are opening this project for the first time

Project title	Online Job Recruitment (JobConnect)
Type	Web application / website
Methodology	Agile (modular folders map to SRS / use cases / test plan)
Frontend	HTML5, CSS3, JavaScript (vanilla — no React/Vue)
Backend	PHP (procedural + helpers, PDO)
Database	MySQL (`database.sql`)
Theme	Professional blue (`#0D47A1`, `#1565C0`, `#1976D2`)
Runs in	Any modern web browser
GitHub	Project repository (clone from your Git remote)

1. What is this project? (Simple explanation)

Imagine a small version of LinkedIn Jobs / Naukri / Indeed. Three types of people use the website:

1. **Candidate** — looks for jobs, uploads resume, applies, tracks status
2. **Employer** — posts jobs, reviews applicants, updates application status
3. **Admin** — controls the whole system (users, job approvals, categories, stats)

This is **not** a React app with separate API servers. It is a classic **PHP website**: when you open a page, PHP talks to MySQL on the server, builds the HTML, and sends the finished page to your browser. That is why the Network tab often shows no XHR/fetch “API” calls — the data is already inside the HTML.

2. How the Frontend works

What is frontend here?

Everything the user sees and clicks in the browser:

- `*.php` pages output HTML structure
- `assets/css/style.css` — colors, layout, responsive design (Flexbox/Grid)
- `assets/js/main.js` — mobile menu, form validation, custom confirm dialogs
- Font Awesome icons + Google Fonts (loaded from CDN)

Main public screens

Page	What it does
<code>index.php</code>	Landing page: hero search, featured jobs, categories, how it works
<code>jobs.php</code>	Browse/search/filter all approved jobs
<code>job-details.php</code>	Full job description, apply, bookmark
<code>login.php</code> / <code>register.php</code>	Sign in / create candidate or employer account

Role dashboards (after login)

Folder	Pages
<code>candidate/</code>	dashboard, profile, applications, bookmarks
<code>employer/</code>	dashboard, profile, post-job, manage-jobs, edit-job, applicants
<code>admin/</code>	dashboard, manage-users, manage-jobs, manage-categories

Shared UI pieces live in `includes/header.php` and `includes/footer.php` so every page looks consistent.

3. How the Backend works

Backend = PHP code that runs on the server before HTML is shown.

File	Job
<code>includes/config.php</code>	Database host, name, user, password, upload limits
<code>includes/db.php</code>	Creates PDO MySQL connection
<code>includes/functions.php</code>	Login checks, flash messages, file upload helpers, CSRF
<code>includes/init.php</code>	Starts session + loads helpers on every page

Request flow (example: Apply to a job)

1. Candidate opens `job-details.php?id=1`
2. PHP loads job from MySQL and shows the page
3. Candidate clicks Apply → form POSTs to same page
4. PHP checks login role, CSRF token, resume exists
5. PHP inserts a row into `applications` table

6. User is redirected to Applications list with a success message

Security built in

- ✓ Passwords hashed with `password_hash()`
- ✓ SQL via PDO prepared statements (helps stop SQL injection)
- ✓ Session login + role checks (candidate/employer/admin)
- ✓ CSRF tokens on important forms
- ✓ Upload validation (PDF resume, image photo/logo, size limits)

4. Database (`database.sql`) explained

`database.sql` is the “blueprint + sample data” file. Import it once into MySQL. It creates database **job_recruitment** and all tables.

Table	Stores
<code>users</code>	Login email, password hash, role, active/blocked
<code>candidates</code>	Job seeker profile, skills, resume file name
<code>employers</code>	Company name, logo, industry, address
<code>categories</code>	Job categories (IT, Sales, HR, ...)
<code>jobs</code>	Job posts (title, salary, location, approval status)
<code>applications</code>	Who applied to which job + status
<code>bookmarks</code>	Jobs saved by candidates

Seed password for demo users: `password`

Demo emails exist only inside the database for testing (login page no longer lists them).

5. Requirements checklist — is the project complete?

Compared with your project brief (Agile web app, HTML/JS/CSS + PHP + MySQL, blue theme):

Required item	Status	Where
Web application / website	COMPLETE	Whole project
Agile-friendly modular structure	COMPLETE	candidate / employer / admin folders
Runs in any browser	COMPLETE	Standard HTML/CSS/JS

Frontend: HTML, CSS, JavaScript	COMPLETE	Pages + assets/
Backend: PHP	COMPLETE	All .php files
Database: MySQL + SQL file	COMPLETE	database.sql
Blue color design	COMPLETE	style.css theme
Candidate register/login/profile/apply/bookmark	COMPLETE	candidate/
Employer post/manage jobs + applicants	COMPLETE	employer/
Admin users/jobs/categories/stats	COMPLETE	admin/
Responsive UI	COMPLETE	CSS media queries
Security (hashing, PDO, sessions)	COMPLETE	includes/
SRS / Gantt / Use-case docs as separate files	NOT IN CODE	Prepare in Word/Draw.io for submission (code is ready to map to them)

Verdict: The software application features are complete and demo-ready. Separate academic documents (SRS, Gantt chart, use-case diagram, test plan) are normally written outside the code and attached to your report — the code structure already supports those diagrams.

6. Full feature summary by role

Candidate

- ✓ Register & login
- ✓ Edit profile (photo, skills, education, experience, PDF resume)
- ✓ Search/filter jobs
- ✓ Apply + track Pending/Shortlisted/Rejected/Selected
- ✓ Bookmark jobs
- ✓ Dashboard stats

Employer

- ✓ Company profile + logo
- ✓ Post job (needs admin approval)
- ✓ Edit / close / delete jobs
- ✓ View applicants & resumes
- ✓ Update applicant status

✓ Dashboard stats

Admin

- ✓ Stats dashboard + charts
- ✓ Block/delete users
- ✓ Approve/reject/close/delete jobs
- ✓ Manage categories

7. How to install & run (for a new person)

Option A — XAMPP (most common for college projects)

1. Install **XAMPP** and start **Apache + MySQL**
2. Copy/clone project into `C:\xampp\htdocs\Job-Connect`
3. Open phpMyAdmin → Import → select `database.sql` → Go
4. Open browser: `http://localhost/Job-Connect/`

```
git clone <your-repo-url>
# then move/copy into htdocs if needed
```

DB settings (usually already correct) in `includes/config.php`:

```
DB_HOST = localhost
DB_NAME = job_recruitment
DB_USER = root
DB_PASS = (empty on default XAMPP)
```

Important: In this project there is *no separate* “npm start frontend” and “npm start backend”.
Apache runs PHP pages. Frontend CSS/JS are just static files served with those pages.

Option B — Docker (if you already have Docker Desktop)

From the project folder:

```
docker compose up -d
```

Then open:

```
http://localhost:8080/
```

Useful Docker commands:

```
# See running containers
docker compose ps

# View web logs
docker compose logs -f web

# Stop everything
docker compose down

# Stop and delete database volume (full reset)
docker compose down -v
docker compose up -d
```

Docker MySQL (from your PC): host `127.0.0.1`, port `3307`, user `root`, password `root`, database `job_recruitment`.

8. How to check `database.sql` and the data

A) Read the SQL file in the project

Open `database.sql` in VS Code / Cursor. You will see:

- `CREATE DATABASE` / `CREATE TABLE` statements
- `INSERT INTO` sample users, jobs, applications

B) Check data in phpMyAdmin (XAMPP)

1. Go to `http://localhost/phpmyadmin`
2. Click database **job_recruitment**
3. Open tables: `users`, `jobs`, `applications`, ...
4. Click **Browse** to see rows

C) Check data with Docker MySQL commands

```
docker exec -it jobconnect-db mysql -uroot -proot job_recruitment

# Inside MySQL:
SHOW TABLES;
SELECT id, email, role, status FROM users;
SELECT id, title, location, status FROM jobs;
SELECT id, job_id, candidate_id, status FROM applications;
EXIT;
```

One-line examples without entering MySQL shell:

```
docker exec jobconnect-db mysql -uroot -proot job_recruitment -e "SHOW TABLES;"
docker exec jobconnect-db mysql -uroot -proot job_recruitment -e "SELECT id,title,location,stat
docker exec jobconnect-db mysql -uroot -proot job_recruitment -e "SELECT id,email,role FROM use
```

9. How to test the website quickly

1. Open home page → search jobs
2. Register a new candidate OR login with seed user
3. Upload resume in profile → apply to a job
4. Login as employer → open Applicants → change status
5. Login as admin → approve a pending job, view charts

Seed login password (database demo users): password

Examples: admin@jobconnect.com · rahul.sharma@email.com · hr@techvista.com

10. Project architecture diagram (simple)

```
Browser (HTML/CSS/JS)
  |
  | HTTP request
  v
Apache + PHP pages
  |
  | PDO SQL
  v
MySQL database
(job_recruitment)
```

11. Folder map (one glance)

Job-Connect/	
admin/	Admin panel
candidate/	Job seeker area
employer/	Company/recruiter area
assets/	CSS, JS, images
includes/	Shared PHP (config, DB, header, footer)
uploads/	User uploaded resumes/photos/logos
database.sql	Schema + seed data
docker-compose.yml	
index.php	Home
README.md	Short setup readme

